Day 13: Declarative Kubernetes: Pod, Deployment & Clus

Writing K8s configurations, Pod & Deployment Spec schemas, and ClusterIP Internal Networking

Module 1: Kubernetes Manifests & Pod Configuration

Kubernetes objects are managed declaratively using configuration files structured in YAML (YAML Ain't Markup Language). Each file describes the desired state of a resource, which the Kubernetes Control Plane continuously works to maintain. All Kubernetes manifests require four root schema keys: apiVersion, kind, metadata, and spec.

1. Understanding Pod Manifests (pod.yml)

A Pod configuration defines a single deployable unit in the cluster. It holds one or more containers, resource limits, port settings, and storage volumes. Below is a detailed example blueprint for a Python Flask Pod.

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: webserver
spec:
  containers:
  - name: flask-container
    image: python-flask-app:latest
    imagePullPolicy: Never
    ports:
    - containerPort: 5000
```

Key Pod Attributes Explained

- labels (under metadata): Key-value pairs (e.g. app: webserver) that are attached to objects. These are critical because other resources (like Services and Deployments) use selectors to target specific Pods.

- imagePullPolicy: Never: Tells Kubernetes not to pull the image from an online registry (like Docker Hub) and to only use the image built locally inside the Node's Docker engine.

- containerPort: Indicates the network port the application inside the container is actively listening on.

Module 2: Managing Workloads at Scale: Deployments

While you can run individual Pods directly, doing so is not recommended in production. If a standalone Pod fails, it is not rescheduled. Deployments solve this by acting as a high-level manager that controls a ReplicaSet, ensuring the desired number of Pod instances are running and handling seamless updates.

1. Understanding Deployment Manifests (deployment.yml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webserver
  template:
    metadata:
      labels:
        app: webserver
    spec:
      containers:
      - name: web-container
        image: python-flask-app:latest
        ports:
        - containerPort: 5000
```

Key Deployment Attributes Explained

- replicas: Specifies the number of identical Pods to run concurrently. In this example, the controller maintains exactly 3 healthy Pods across the cluster worker nodes.

- selector.matchLabels: Tells the Deployment controller which Pods to manage. The key-value pairs listed here must exactly match the labels specified inside the Pod template below it.

- template: The blueprint containing Pod metadata and specifications that the Deployment uses when creating new Pods.

Module 3: Internal Networking with ClusterIP Services

Pods in Kubernetes are ephemeral; they are frequently destroyed, recreated, and scaled. Every time a Pod restarts, it receives a new IP address, making direct pod-to-pod communication unreliable. A Service acts as a stable gateway with a permanent IP address and DNS name, routing traffic down to backend Pods matching its label selector.

1. ClusterIP Service Manifest (service.yml)

ClusterIP is the default service type in Kubernetes. It allocates a stable IP address that is reachable only from within the Kubernetes cluster. It is typically used for internal application links, such as connecting an API backend to an internal database.

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: webserver
  ports:
  - protocol: TCP
    port: 80
    targetPort: 5000
```

Key Service Networking Parameters

- type: ClusterIP: Specifies that the service should receive a cluster-internal virtual IP, isolating it from public internet traffic.

- port: The port number that the service exposes inside the cluster. Internal apps will send requests to this port (e.g. http://web-service:80).

- targetPort: The actual port that the backend application container is listening on (5000). The service routes traffic from port 80 down to container port 5000.

- Service Selector Link: The Service matches the selector label 'app: webserver' against all Pod labels. Any incoming request to the ClusterIP is automatically load-balanced across the matching Pods.